



虚拟地理环境

项目报告书

《洪线之上》——基于 Unity 的
地形格网洪水演进、防洪决策与灾损评估交互系统

专 业： 地理信息科学

年 级： 2023 级

任课老师： 江辉仙

2026 年 6 月 17 日

目录

第一章 项目概述 1

第二章 需求分析 2

第三章 系统总体设计 4

第四章 数据准备与 GIS 处理 5

第五章 关键技术实现 10

第六章 游戏玩法设计 15

第七章 开发过程 16

第八章 开发难点、问题修复与参数迭代 27

第九章 测试与验证 29

第十章 项目成果与价值 31

第十一章 总结 32

第十二章 小组分工与协作 32

第十三章 小组成员心得体会 35

第一章 项目概述

1.1 项目名称与定位

项目名称为《洪线之上》。该名称强调洪水风险中的临界水位、防线布设与人在灾害情境中的主动决策。“洪线”既指地图与三维场景中的洪水边界，也指玩家在有限预算下建立的防线；“之上”则强调通过观察、规划和交互，使玩家不只是被动观看灾害结果，而是在洪水到来前主动寻找更优防洪策略。

本项目是一个基于 Unity 的虚拟地理洪水风险模拟游戏，主题是在真实地形语境下进行防洪规划、堤坝布设、洪水演进观察和灾损评估。系统将地形格网、洪水蔓延、防洪工程、第一人称巡查、任务引导、风险 HUD 和 Windows 桌面交付整合为一个可演示、可运行、可用于课程汇报的完整交互系统。

1.2 项目背景

洪水灾害具有明显的空间传播特征：地形高低、河道位置、村庄分布、农田范围和防洪工程共同影响淹没路径与损失结果。传统静态地图能够表达位置与范围，但难以直观呈现洪水逐步上涨、扩散、受阻和绕流的过程。

虚拟地理系统适合把真实地理数据、三维场景和交互机制结合起来，使学习者能够以第一人称或飞行视角观察地形格局，并通过布设防线理解防洪决策的空间含义。《洪线之上》正是在这一背景下形成的课程项目原型。

从开发角度看，本项目并不追求工业级二维水动力仿真，而是将重点放在“可解释、可交互、可演示”的洪水风险认知过程上。项目用离散格网表示地形单元，用水位阈值和相邻格传播表示洪水扩散，用网格边阻挡表示大坝作用，并用 HUD 与损失统计将模拟结果转化为玩家能够理解的反馈。

1.3 建设目标

- 构建包含地形、村庄、农田、河岸、道路、植被和水体表现的三维地理场景。
- 实现从指定源头开始、随水位上涨逐步扩散的洪水模拟过程。
- 实现多类型大坝布设、长坝预览、连续旋转、预算约束和网格边阻挡机制。
- 实现任务提示、风险 HUD、损失评估和水下视觉反馈，让玩家理解防洪效果。

- 形成可运行、可演示、可打包交付的 Windows 桌面程序。
- 完整开发过程记录，说明从数据处理、程序实现、调试到打包的技术路线。

1.4 最终成果

包括 Unity 工程、主场景、Windows 程序、构建脚本等。



图 1 最终运行画面：开始菜单

第二章 需求分析

2.1 用户需求

目标用户需要在可视化环境中观察地形与洪水风险对象，尝试不同防线策略，并通过实时反馈判断防洪效果。系统需要让玩家在较短时间内理解“地形影响洪水路径”“工程改变扩散边界”“预算限制带来取舍”这三个核心概念。

- 观察地形、村庄、农田、河道与低洼区的空间关系。
- 在洪水到来前选择坝型、旋转长坝并布设防线。
- 在预算约束下完成若干条关键防线。
- 启动洪水并观察水位上涨、蔓延方向和防线阻挡效果。

- 查看村庄、农田、预算、防线状态和综合损失反馈。
- 在演示中能够快速实现“布设前—洪水启动—防线阻挡—损失评估”的闭环。

2.2 功能需求

模块	需求描述	开发关注点
虚拟地理场景	呈现地形起伏、村庄、农田、河岸、道路、植被、水体和环境氛围。	保证地形尺度、对象位置和视角体验便于观察。
洪水模拟	从固定源头启动，按照逻辑格网随水位上涨逐步扩散，并受地形和阻挡边约束。	不做高成本流体仿真，重点保证可观察的渐进扩散和可解释的传播规则。
防洪工程	支持沙袋、土堤、混凝土堤、导流堤和闸门等坝型，支持长坝预览与建造。	将可见坝体与隐藏网格阻挡边绑定，避免只有模型没有逻辑效果。
玩家控制	支持第一人称移动、观察、冲刺、跳跃和飞行模式。	兼顾地面巡查与汇报时的全局观察。
HUD 与任务	显示预算、坝型、洪水状态、防线状态、风险评估、任务阶段和帮助面板。	避免信息重叠，保证关键参数在演示时可以一眼读出。
损失评估	统计淹没范围、村庄安全、农田影响、防线成本和任务结果。	将模拟状态转化为更像游戏任务结果的反馈。
桌面交付	通过 WindowsBuild 脚本构建可运行程序。	保证离开 Unity 编辑器后仍能独立运行。

2.3 非功能需求

- 运行稳定：洪水扩散采用可控的格网传播模型，避免高成本流体仿真导致性能不可控。
- 交互清晰：HUD 分区布置，避免遮挡和重叠，关键状态以短标签呈现。
- 参数可解释：源头、水深、扩散速度、预算和长坝长度均采用最终验证参数。
- 演示友好：玩家出生点靠近洪水源头，远裁剪距离增大，便于汇报展示。
- 交付完整：工程、主场景、构建脚本和 Windows 可执行程序形成闭环。
- 过程可追溯：每个主要机制都能够说明开发原因、实现、调试问题和取舍。

第三章 系统总体设计

3.1 总体架构

系统采用“GIS 数据准备 - 地形格网 - Unity 场景 - 洪水模拟 - 大坝阻挡 - HUD 与评估 - Windows 交付”的流程组织。ArcGIS Pro 负责前期 DEM 与遥感影像处理，Unity 负责三维场景、输入交互、实时逻辑和桌面构建。

架构链路：ArcGIS Pro 数据处理→DEM / 遥感影像 / ASCII Grid→Unity 逻辑格网与三维场景→洪水 frontier 扩散→大坝网格边阻挡→HUD / 损失评估→Windows 桌面程序。

该架构的重点是把地理数据和游戏逻辑放在同一套坐标关系中。DEM 决定地形高度，ASCII Grid 决定洪水扩散的逻辑单元，Unity 场景对象决定玩家可见的村庄、农田、河岸和大坝。开发过程中最关键的工作，是让三维显示、逻辑格网、玩家操作和 HUD 反馈保持一致。

3.2 模块划分

层级	组成内容	作用
数据层	DEM、遥感影像、ASCII Grid、世界文件	提供地形高程、纹理参考和空间对齐依据。
场景层	地形、村庄、农田、河岸、道路、植被、水体	构成可巡查、可观察的虚拟地理环境。
模拟层	洪水扩散、大坝阻挡、溃堤放水、闸门通断	决定洪水是否能到达相邻格、防线是否有效。
交互层	第一人称移动、飞行观察、长坝预览、旋转建造	提供玩家决策和空间操作能力。
反馈层	HUD、任务阶段、风险评估、损失统计、水下效果	把模拟状态转化为玩家可理解的信息。
交付层	batch validation、WindowsBuild、可执行程序	保证最终系统可验证、可运行、可汇报。

3.3 核心体验链路

1. 进入场景，观察地形、村庄、农田和河道关系。
2. 根据 HUD 和任务提示判断重点保护区域。
3. 选择坝型，打开长坝预览，旋转并布设防线。
4. 在预算约束下完成防洪规划。

5. 启动洪水，观察水位上涨、洪水蔓延和大坝阻挡效果。
6. 查看风险评估、损失反馈、预算消耗和任务结果。

3.4 设计取舍

本项目在设计时进行了明确取舍：一方面保持地理数据与空间对象的真实感，另一方面避免过度追求复杂水动力计算。原因是课程汇报场景更需要稳定、直观、可复现的交互体验。如果使用完整二维水动力模型，虽然物理真实性更强，但开发周期、性能消耗和调参难度都会显著增加，不利于最终交付。

因此，项目采用逻辑格网模型作为折中方案：格网高程控制是否可淹没，frontier 队列控制洪水边界推进，阻挡边控制大坝效果，HUD 控制反馈表达。该方案能够清晰地体现“水从低处扩散”“坝体改变传播路径”“缺口会继续放水”等核心现象，同时便于在 Unity 中完成实时交互和桌面打包。

第四章 数据准备与 GIS 处理

4.1 数据准备目标

《洪线之上》的核心玩法建立在真实地形格网与三维地理场景之上，因此在进入 Unity 开发之前，必须先完成 GIS 数据准备。数据处理的目标并不是单纯制作一张地图，而是为后续的洪水演进、防洪设施布设、损失评估和三维可视化提供统一的数据基础。

本项目的数据准备主要服务于三个方面：第一，利用 DEM 数字高程模型构建真实地形起伏，使洪水扩散能够受到地势高低影响；第二，利用遥感影像或地表纹理增强场景真实感，使玩家能够在三维环境中直观识别河岸、村庄、农田、道路和植被等地理要素；第三，将处理后的 DEM 转换为 Unity 可读取的逻辑格网，使每一个格网单元都能够参与洪水模拟、淹没判断和防洪工程阻挡判断。

因此，本章重点说明 DEM、遥感影像、研究区范围、投影坐标系、裁剪重采样、三维验证、ASCII Grid 导出以及 Unity 导入之间的完整处理流程。

4.2 必要数据与数据源选择

本项目最核心的数据是 DEM 数字高程模型。DEM 以栅格形式记录地表每一个网格单元的高程值，能够反映研究区内的地形起伏、坡度变化、低洼区域、河谷

走向和高地分布。洪水是否能够继续向相邻格扩散，目标区域是否会被淹没，防洪大坝是否布设在关键位置，都与 DEM 所提供的地形高程有关。因此，DEM 数据是整个项目中最基础、最不可替代的数据。

DEM 数据可以选择地理空间数据云提供的 GDEM V3 30m 数据，也可以选择 ALOS 12.5m DEM。GDEM V3 数据获取方便，适合课程设计和快速原型开发；ALOS 12.5m DEM 分辨率更高，能够更精细地表达小范围地形起伏，更适合本项目 $5\text{km} \times 5\text{km}$ 的虚拟地理场景。最终项目运行时采用约 12.5m 单元格的逻辑格网，与 400×400 的格网规模相匹配。

除 DEM 之外，遥感影像主要用于提升视觉真实感。影像可以来自 Sentinel-2 真彩色波段，也可以使用天地图、自然资源卫星遥感云服务平台、ArcGIS Pro Living Atlas 或其他公开影像服务。影像数据本身不直接决定洪水扩散逻辑，但它能够作为 Unity 地形贴图，使三维地形不再只是单一颜色或程序化纹理，而是具有接近真实地表的视觉效果。

最终项目中，村庄、农田、河岸、道路等对象主要作为虚拟地理场景要素与损失评估对象参与系统逻辑。

4.3 研究区范围确定

项目研究区设置为约 $5\text{km} \times 5\text{km}$ 的小范围区域。选择这一范围的原因是：一方面， $5\text{km} \times 5\text{km}$ 足以包含河道、农田、村庄、道路和地形高差等基本地理要素，能够支撑洪水风险模拟；另一方面，该范围不会导致格网数量过大，适合 Unity 实时运行和课堂演示。

在 ArcGIS Pro 中确定研究区时，需要优先使用投影坐标系，而不是经纬度坐标系。因为项目需要精确控制 $5\text{km} \times 5\text{km}$ 的范围，并且 Unity 中的世界坐标也以米作为主要度量单位。如果直接使用经纬度坐标，不利于距离、面积和格网尺寸的统一计算。

研究区边界可以通过两种方式确定。第一种方式是在 ArcGIS Pro 中新建本地面要素类，使用矩形绘制工具绘制一个 $5000\text{m} \times 5000\text{m}$ 的矩形区域；第二种方式是使用“创建渔网”工具，设置 1 行 1 列、像元宽度和高度均为 5000m，从而生成标准的矩形研究区。第二种方式更加精确，能够避免手动绘制带来的边长误差和方向偏差。

完成研究区边界后，需要检查边界图层与 DEM 数据是否使用相同投影坐标系。如果边界与 DEM 坐标系不一致，后续裁剪时可能出现范围偏移、图层无法选择或输出为空等问题。

4.4 DEM 裁剪与重采样

DEM 裁剪的目的是从原始大范围高程数据中提取项目所需的 $5\text{km} \times 5\text{km}$ 区域。ArcGIS Pro 中常用两种裁剪方法：一种是“按掩膜提取”，适合使用任意形状的研究区边界；另一种是“裁剪栅格”，适合矩形范围，处理效率较高。

在使用“按掩膜提取”时，需要将 DEM 作为输入栅格，将研究区边界要素作为掩膜数据，并在环境设置中将处理范围和捕捉栅格设置为原始 DEM。这样可以保证裁剪结果与原始 DEM 像元保持对齐，避免后续格网错位。

在使用“裁剪栅格”时，可以直接选择矩形研究区作为输出范围，也可以手动输入 Xmin、Ymin、Xmax、Ymax 四个坐标值。该方法适合规则矩形研究区，步骤更直接，输出结果也便于后续转换为 Unity 逻辑网格。

裁剪完成后，需要根据项目需要进行重采样。前期通用方案中可以将 $5\text{km} \times 5\text{km}$ 区域重采样为 20m 分辨率，对应 250×250 个格网单元，共 62500 个单元，适合快速测试和性能较弱设备运行。最终项目采用约 12.5m 单元格，对应 400×400 个逻辑格网单元，共 160000 个单元，能够在保持可运行性能的同时提供更细致的洪水扩散表现。

重采样方法可以选择双线性插值或三次卷积。DEM 是连续高程数据，使用双线性插值能够获得较平滑的地形结果；如果需要更平滑的视觉效果，也可以使用三次卷积。处理完成后，应检查输出 DEM 是否存在 NoData 区域、异常高程值或范围缺失。

4.5 遥感影像获取与处理

遥感影像用于 Unity 地形纹理，因此需要尽量选择云量较低、覆盖完整、空间分辨率较高的影像数据。若使用 Sentinel-2，可选择 L2A 级别数据，并下载 B4、B3、B2 三个可见光波段，用于合成真彩色影像。若使用天地图或其他在线影像服务，也可以在 ArcGIS Pro 中加载服务图层后导出目标区域影像。

影像处理的关键是保证影像与 DEM 使用相同的坐标系和相同的空间范围。首先，将遥感影像加载到 ArcGIS Pro 工程中，检查其空间参考信息；如果影像与

DEM 坐标系不同，需要使用“投影栅格”工具统一到相同的 UTM 投影坐标系。其次，使用研究区边界对影像进行裁剪，使影像范围与 DEM 裁剪结果一致。最后，将裁剪后的影像导出为 JPG 或 PNG 格式，并保留对应的世界文件，例如 .jgw 或 .pgw。

JGW 世界文件记录了影像像素大小、旋转参数和左上角坐标等信息。虽然 Unity 不会像 ArcGIS Pro 那样自动读取世界文件，但在进行纹理对齐、空间换算和数据归档时，JGW 文件能够提供重要参考。它可以帮助开发者确认影像贴图的真实地理范围，避免影像与地形出现偏移或缩放错误。

4.6 DEM 与影像叠加验证

在数据导入 Unity 之前，需要先在 ArcGIS Pro 中进行三维可视化验证。验证的目的包括：检查 DEM 是否能正确表达地形起伏，检查遥感影像是否与 DEM 范围一致，检查影像贴地后是否出现偏移、拉伸、黑边或遮挡问题。

具体操作是在 ArcGIS Pro 中新建局部场景，将裁剪后的 DEM 添加为地面高程源。需要注意的是，DEM 添加为高程源后只负责提供地形起伏，不一定会作为普通图层显示。如果需要观察 DEM 本身的色彩或阴影效果，还需要将 DEM 再次作为普通图层添加到场景中，并设置山体阴影或高程着色符号系统。

随后，将裁剪好的遥感影像添加到三维场景中，并将其高程设置为“在地面上”，使影像贴合 DEM 地形起伏。为了增强视觉效果，可以适当调整垂直夸张系数，平坦区域可设置为 2 到 3 倍，使地形起伏更容易被观察。在验证过程中，如果出现影像与 DEM 偏移，通常是坐标系不一致或裁剪范围不一致导致；如果出现黑边或空洞，则需要检查 NoData 设置和影像裁剪结果。

4.7 逻辑格网导出

Unity 中的洪水模拟需要直接读取二维高程矩阵，因此 DEM 需要导出为适合程序解析的逻辑格网格式。相比图片格式，ASCII Grid（.asc）更适合作为洪水模拟的输入文件。ASCII Grid 是纯文本格式，文件头中包含列数、行数、左下角坐标、像元大小和 NoData 值，后续内容为规则排列的高程矩阵，能够直接对应 Unity 中的二维数组。

本项目最终运行时使用的逻辑格网文件为 logical_grid_5km.asc。系统通过 VGeoGridManager 脚本读取该文件，解析网格行列数、单元大小和高程值，并为

每一个格子建立运行时状态。每个格子不仅保存高程信息，还会记录是否被淹没、是否存在阻挡边、是否与防洪设施相关等逻辑属性。

ASCII Grid 导出可通过 ArcGIS Pro 的“栅格转 ASCII”工具完成。导出时需要保证 DEM 已经完成裁剪、重采样和投影统一。若输出过程中出现“无法使用指定的像素类型、波段数或色彩映射表以指定格式创建输出栅格”等问题，应检查输入数据的像素类型和波段数。DEM 推荐导出为单波段高程数据，逻辑格网优先使用 .asc 格式；遥感影像则应保留 RGB 三波段，并导出为 JPG 或 PNG 纹理。

4.8 Unity 导入格式与空间对应

ArcGIS Pro 处理完成后，需要向 Unity 工程导入三类主要数据：一是 DEM 高度信息，用于生成地形起伏或逻辑格网；二是遥感影像纹理，用于覆盖地表并增强真实感；三是研究区范围、世界文件或辅助矢量信息，用于校准空间位置。

在 Unity 中，DEM 可以有两种用途。第一种用途是导出为 16 位 PNG 高度图，用于生成 Terrain 地形；第二种用途是导出为 ASCII Grid，用于洪水模拟逻辑。前者偏向视觉表现，后者偏向程序计算。本项目更强调洪水扩散、坝体阻挡和损失评估，因此 ASCII Grid 是更关键的运行时数据。

由于 Unity 世界坐标不直接使用真实地理坐标，需要将 ArcGIS Pro 中的大坐标值转换为局部坐标。常见做法是以研究区左下角作为局部原点，将所有坐标减去左下角坐标值，再映射到 Unity 场景中。这样可以避免直接使用过大的 UTM 坐标导致浮点精度问题，也方便玩家移动、洪水扩散和大坝布设逻辑计算。

最终项目中，格网行列与 Unity 世界坐标之间通过 VGeoGridManager 进行映射。洪水源头、玩家出生点、大坝布设位置和淹没格网都基于这一坐标转换关系运行。只有当前期 GIS 数据范围、像元大小和 Unity 坐标映射保持一致时，洪水模拟结果才会与三维场景视觉表现对应起来。

4.9 数据处理中的问题与解决

在 ArcGIS Pro 数据处理过程中，最常见的问题是坐标系不一致。DEM、遥感影像和研究区边界必须使用相同的投影坐标系，否则可能出现裁剪范围无效、影像偏移、图层无法选择、三维叠加错位等问题。本项目优先使用以米为单位的 UTM 投影坐标系，以保证 $5\text{km} \times 5\text{km}$ 范围和格网大小能够精确表达。

第二类问题是图层不可编辑或掩膜不可用。裁剪栅格时，掩膜图层本身并不一定需要处于编辑状态，但它必须是有效的面要素，并且需要与输入 DEM 存在空间重叠。如果网络图层或只读图层无法直接使用，可以将其导出为本地 Shapefile 或要素类后再进行裁剪。

第三类问题是影像裁剪进度为零或输出失败。这通常与输出路径、坐标系、裁剪范围和输入数据完整性有关。输出路径应尽量避免中文、空格和特殊符号，裁剪范围必须与影像实际范围重叠，并且输出文件不能被其他程序占用。

第四类问题是 DEM 添加为高程源后没有显示。其原因在于高程源只提供地形起伏，并不等同于普通可见图层。如果需要显示 DEM 的灰度、色彩或山体阴影，需要将 DEM 另行添加为普通图层，并设置符号系统。

第五类问题是 Unity 中影像贴图与地形不匹配。该问题通常来自影像与 DEM 的范围不一致、像元大小不一致或坐标系不一致。解决方法是在 ArcGIS Pro 中完成统一投影、统一裁剪范围，并保留 JGW 世界文件作为对齐检查依据。

4.10 本章小结

通过 ArcGIS Pro 的数据准备，本项目完成了从真实地理数据到 Unity 运行数据的转换流程。DEM 数据提供洪水模拟所需的地形基础，遥感影像提供三维场景的视觉纹理，研究区边界控制项目空间范围，ASCII Grid 则将高程数据转化为可被 C# 脚本解析的逻辑格网。

与早期 20m、250×250 格网的通用处理方案相比，最终项目统一采用 400×400、约 12.5m 单元格的运行时格网描述。该参数能够更好地表达 5km×5km 研究区内的地形细节，并与后续洪水扩散、大坝阻挡、HUD 显示和损失评估模块保持一致。通过这一流程，项目实现了 GIS 数据处理、三维地理表达和游戏逻辑开发之间的衔接。

第五章 关键技术实现

5.1 Unity 与 C# 运行框架

项目主体采用 Unity 6000.0.3f1 和 C# 脚本实现。Unity 负责场景管理、渲染、输入、音效、碰撞、粒子、水面表现和 Windows 桌面打包；C# 脚本负责逻辑格网、洪水扩散、大坝建造、HUD、任务阶段和评估统计。

涉及 Unity 包和模块包括 `com.unity.ugui`、`com.unity.visualscripting`、`com.unity.timeline`，以及 Unity Physics、Particles、Terrain、Audio、UI、Video、XR 等内置模块。实际开发中，核心逻辑集中在 `Assets/VGeo` 目录下，便于课程汇报时说明模块边界，也便于后续维护和打包。

5.2 最终关键参数

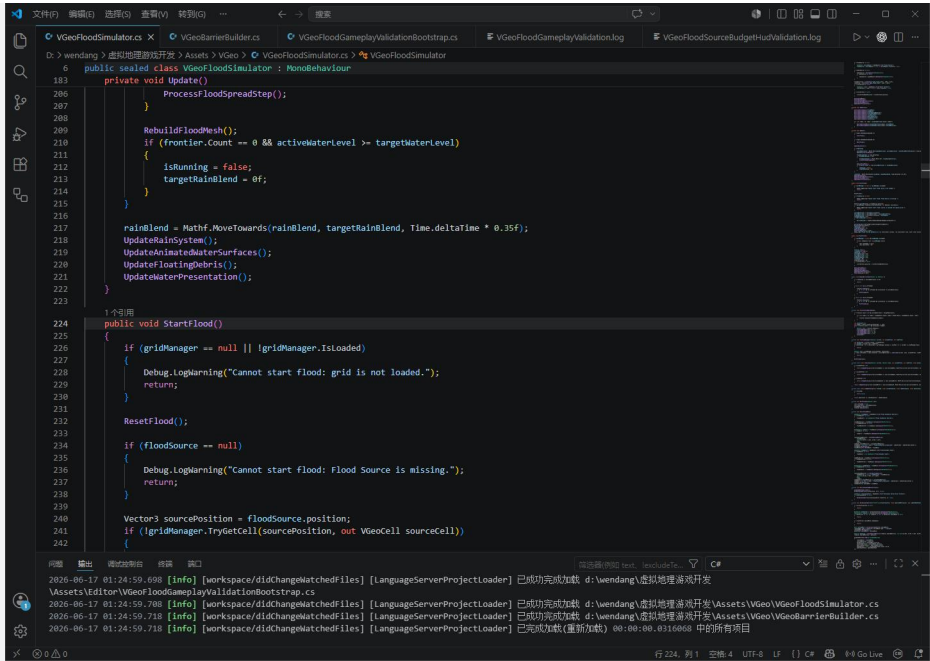
模块	最终参数
Unity 版本	Unity 6000.0.3f1
洪水源头格网	列 234, 行 142
洪水源头世界坐标	约 (437.5, 71.0, 725.0)
洪水深度	5m
水位上涨速度	0.45m/s
洪水扩散批量	18 cells/frame
洪水扩散间隔	0.08s
初始预算	2500
单条长坝长度	30 格, 约 375m
玩家移动速度	44
飞行垂直速度	26
相机远裁剪	24000
玩家出生逻辑	Flood Source + (45, 0, -55) 附近贴地

5.3 洪水逻辑算法

洪水模拟采用面向交互演示的离散格网扩散模型，而不是完整二维水动力模型。系统从 Flood Source 所在格开始，以源头高程为基准，将目标水位设置为源头高程加 5m。水位按 0.45m/s 上涨，洪水按 0.08s 间隔分批向相邻格扩散，每批最多处理 18 个格。

- 洪水源头缺失时拒绝启动，不回退到世界原点。
- 洪水只扩散到当前水位可淹没的相邻格。
- 每次扩散都会检查两格之间的网格边是否被堤坝阻挡。

- 大坝溃堤后，缺口两侧会重新加入洪水 frontier，使水从缺口继续扩散。
- 水位上涨和格网扩散分离：水位负责“能淹到多高”，frontier 负责“从哪里向外扩散”。



```
public sealed class VGeoFloodSimulator : MonoBehaviour
{
    private void Update()
    {
        ProcessFloodSpreadStep();
    }

    RebuildFloodMesh();
    If (frontier.Count == 0 && activeWaterLevel >= targetWaterLevel)
    {
        isRunning = false;
        targetRainBlend = 0f;
    }

    rainBlend = Mathf.MoveTowards(rainBlend, targetRainBlend, Time.deltaTime * 0.35f);
    UpdateRainSystem();
    UpdateEliminatedWaterSurfaces();
    UpdateFloatingDebris();
    UpdateWaterPresentation();
}

1个引用
public void StartFlood()
{
    if (gridManager == null || !gridManager.IsLoaded)
    {
        Debug.LogWarning("Cannot start flood: grid is not loaded.");
        return;
    }

    ResetFlood();

    if (floodSource == null)
    {
        Debug.LogWarning("Cannot start flood: Flood Source is missing.");
        return;
    }

    Vector3 sourcePosition = floodSource.position;
    if (!gridManager.TryGetCell(sourcePosition, out VGeoCell sourceCell))
    {
    }
}
```

图 2 洪水扩散代码

5.4 大坝与防洪系统

大坝系统是《洪线之上》的核心交互机制。玩家通过 U 打开长坝预览，按住 C 并移动鼠标实现 360 度连续旋转，左键确认布设。视觉上，一条长坝是一整条连续实体；逻辑上，系统会把长坝覆盖范围转换为多个隐藏的网格阻挡边。

坝型	作用定位	开发实现要点
Sandbag 沙袋	成本低，可短时阻挡，低耐久，支持溃堤后放水。	保留可损坏状态，溃堤后释放阻挡边并重新激活缺口附近 frontier。
Earth 土堤	中等成本与中等防护能力。	用于体现比沙袋更稳但不如混凝土可靠的工程方案。
Concrete 混凝土堤	当前 5m 洪水玩法下的可靠主防线，不会被常规洪水自动摧毁。	启动前刷新阻挡边，有效坝顶高于目标洪水水位。
GuideBank 导流堤	用于导流和路径控制。	强调改变洪水传播路径，而不只是直接封堵。
FloodGate 闸门	可开闭，用于控制水流通断。	通过状态切换决定阻挡边是否参与扩散检查。

- 大坝会把逻辑格网边标记为 **blocked**。
- 洪水扩散时，如果相邻两格之间存在有效阻挡边，则不能直接穿过。
- 混凝土大坝在洪水启动前会刷新阻挡边，避免洪水首帧越过大坝。
- 混凝土大坝坝顶保证高于本轮目标洪水水位。
- 沙袋保留可损坏和强制溃堤后放水的表现，用于体现不同工程材料的差异。
- 单条长坝已加长到 30 格，约 375m，能够覆盖更大区域。

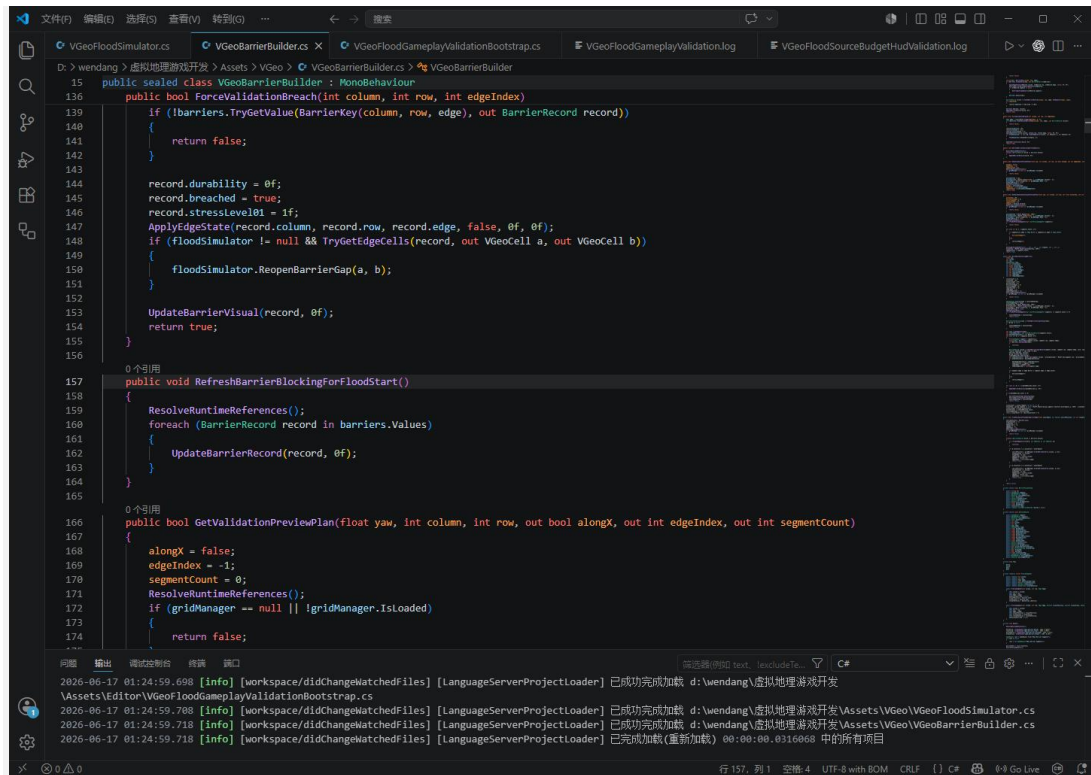


图 3 大坝阻挡代码

5.5 HUD、任务与风险反馈

最终 HUD 已经过重排，目标是避免遮挡和信息重叠。左侧显示地形格网、洪水状态和防线规划；右侧显示任务总览、阶段提示和风险评估；底部居中显示水下状态；Tab 帮助面板默认隐藏，需要时打开。

- 显示当前预算和已用预算。
- 当前坝型和长坝成本估算。

- 显示洪水状态、水位和淹没格网数。
- 显示防线数量、施工状态、危险状态和闸门状态。
- 显示损失评估、任务阶段、村庄 / 农田风险反馈。

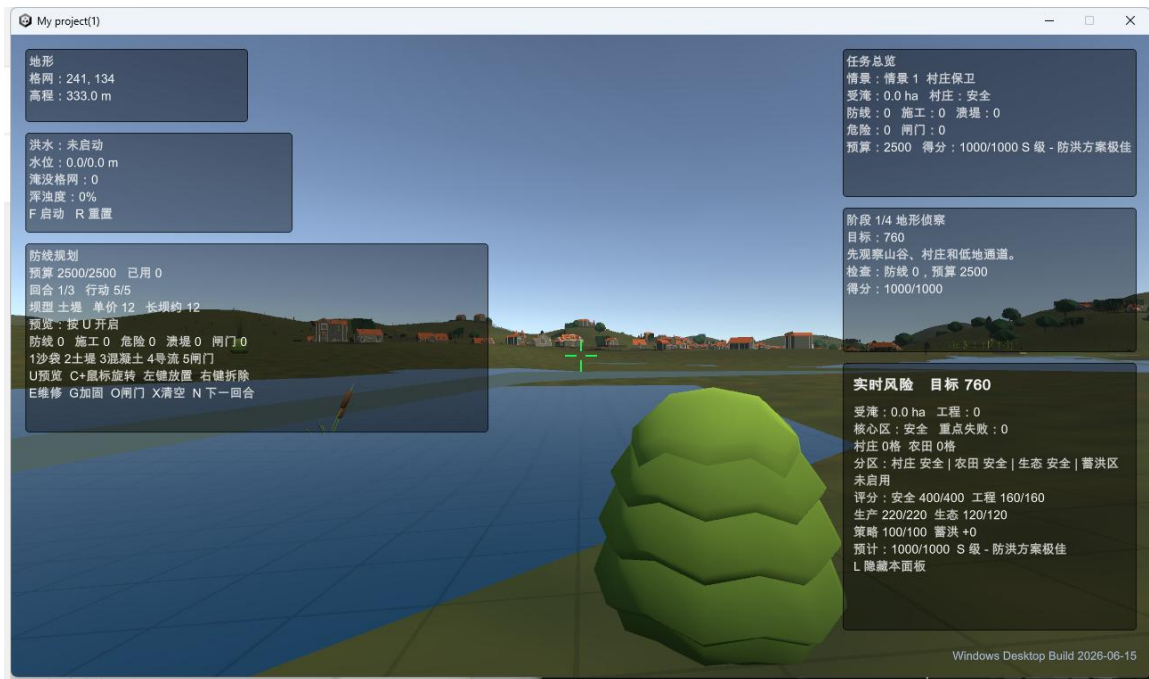


图 4 最终 HUD

5.6 损失评估与阶段管理

系统包含任务阶段、损失评估和场景统计逻辑，用于将“是否挡住洪水”转化为可解释的结果反馈。评估内容包括淹没范围、村庄安全状态、农田受影响情况、防线建设成本、大坝状态与溃堤信息，以及综合任务反馈和分数倾向。

阶段管理的意义在于让系统从单纯模拟变成一个完整任务。玩家不是随意在场景里摆放物体，而是在“观察风险—布设防线—启动洪水—查看结果”的顺序中完成一次防洪规划练习。

第六章 游戏玩法设计

6.1 玩家控制

操作	功能	设计目的
WASD	移动	支持地面巡查和靠近风险对象。
鼠标	观察	控制视角，观察地形起伏与洪水传播方向。
Shift	冲刺	提高大范围场景的移动效率。
Space	跳跃；双击切换飞行模式	兼顾地面体验和汇报展示需要。
飞行模式 Space / Ctrl	上升 / 下降	便于从高处观察洪水范围和大坝布设效果。
U	打开长坝预览	在建造前判断长度、方向和覆盖范围。
按住 C + 鼠标移动	连续旋转长坝	解决固定方向不便布设的问题。
左键	确认布设	完成从预览到实体与逻辑阻挡边的转换。
F	启动洪水	触发水位上涨和 frontier 扩散过程。
Tab	显示 / 隐藏帮助面板	减少 HUD 常驻遮挡。

6.2 决策机制

玩法围绕防洪规划展开。玩家需要在洪水启动前观察地形与风险对象，选择合适的坝型，并在预算限制下布设防线。不同坝型在成本、耐久、阻挡能力和功能定位上存在差异，玩家需要在保护村庄、减少农田损失和控制预算之间做出取舍。

预算机制的作用不是单纯增加难度，而是让玩家不能无限堆砌混凝土堤。2500 的初始预算能够支持玩家建设关键防线，但需要优先考虑源头、低洼通道、村庄入口和农田边界等位置。

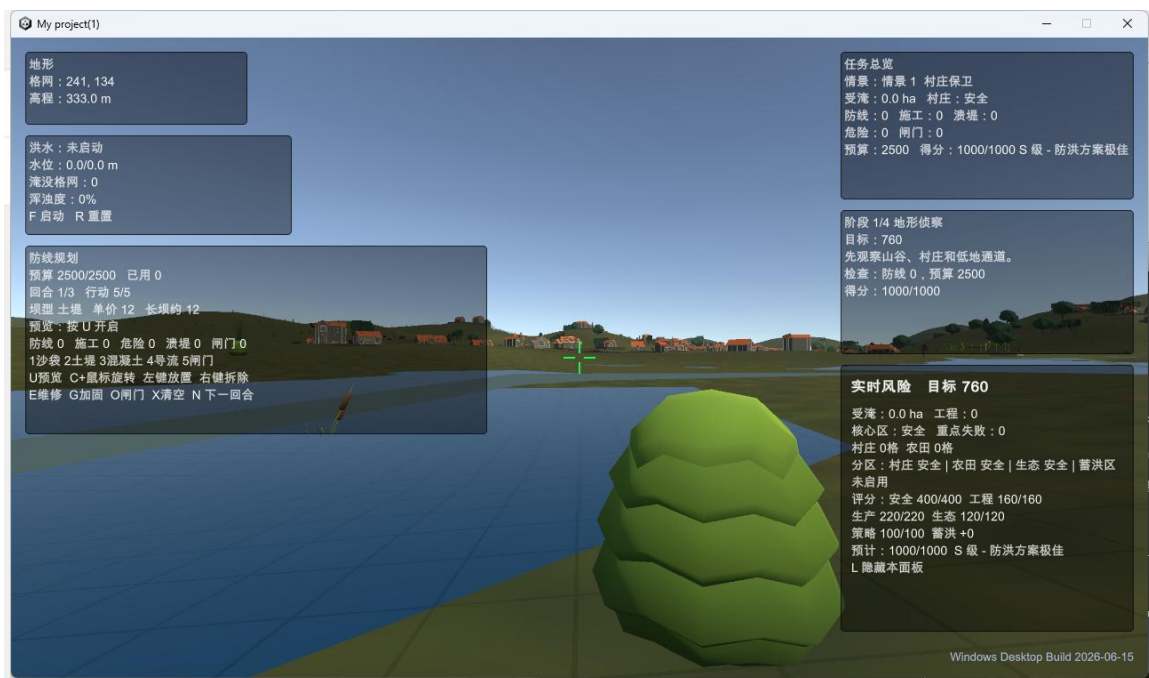


图 5 建造前规划状态

6.3 反馈闭环

洪水启动后，系统通过水位上涨、淹没格网、坝体阻挡、溃堤、风险面板和损失统计形成反馈闭环。玩家不仅能看到水是否越过防线，还能理解不同工程策略对洪水路径和灾损结果的影响。

如果防线有效，洪水会在阻挡边前停滞或绕行；如果防线存在缺口，frontier 会从缺口继续扩散；如果沙袋被破坏，缺口附近格网会重新进入传播队列。这些反馈让玩家能够从结果反推布设策略是否合理。

第七章 开发过程

7.1 开发总体路线

本项目的开发按照“先跑通核心逻辑，再补充交互，再优化表现，最后稳定交付”的方式逐步推进。开发早期重点解决地形格网和洪水扩散能否正常工作；中期重点解决玩家如何建坝、坝体如何真正挡水；后期重点解决 HUD 是否清晰、参数是否统一、演示是否稳定以及 Windows 程序能否独立运行。

项目经历了原型验证、数据接入、算法迭代、交互增强、表现优化、测试打包六个主要阶段。每个阶段都围绕一个可验证目标推进：地形能否生成、洪水能否启动、大坝能否阻挡、玩家能否操作、HUD 能否解释状态、构建能否成功。

阶段	主要目标	形成成果	后续影响
原型验证	确定洪水风险与防洪决策主题，明确课程演示方向。	确定《洪线之上》项目定位和核心玩法。	后续所有机制围绕“洪水扩散 + 防线布设 + 损失反馈”展开。
数据接入	完成 DEM、影像、ASCII Grid 的导入和检查。	形成 400 x 400、约 12.5m 单元格的逻辑格网。	为洪水扩散、高程判断和空间定位提供基础。
算法迭代	让洪水从固定源头按水位逐步扩散。	形成 frontier 扩散、水位上涨和分批处理机制。	保证洪水过程可观察、可控、可演示。
交互增强	让玩家能够建造、旋转和选择不同类型大坝。	形成长坝预览、预算控制和网格边阻挡系统。	把项目从“看模拟”变成“做决策”。
表现优化	完善场景、HUD、风险提示、音效、水体和水下反馈。	形成更完整的严肃游戏体验。	提高汇报展示时的可理解性和完成度。
测试交付	自动验证关键机制并打包 Windows 桌面程序。	Build Finished, Result: Success.	证明项目离开编辑器后仍可运行。

7.2 阶段一：选题、命名与目标收敛

最初选题围绕“虚拟地理游戏开发”展开。相比单纯制作地形漫游或静态三维场景，洪水风险主题更适合体现地理学的空间分析特征：洪水不是孤立对象，而是会受到地形、河道、村庄、农田和工程设施共同影响的动态过程。

选题确定后，项目目标被收敛为三个层次：第一层是地理场景可视化，即让玩家能够看到一个有地形、有河岸、有村庄和农田的虚拟环境；第二层是灾害过程模拟，即洪水能够从源头逐步扩散而不是直接显示一片水面；第三层是防洪决策交互，即玩家能够在预算限制下布设大坝，并通过结果判断策略有效性。

项目命名从泛化表达调整为《洪线之上》，是为了突出“临界线”和“主动防线”两个意象。名称本身服务于汇报表达：它比直接叫“洪水模拟系统”更有作品感，也更容易解释游戏目标。

目标问题	开发回答
玩家进入场景后要做什么？	观察地形与风险对象，判断洪水可能路径，并在洪水启动前布设防线。
系统最重要的动态效果是什么？	洪水从固定源头逐步上涨、扩散、受阻或缺口继续传播。
游戏性来自哪里？	来自预算限制、坝型差异、空间布设和最终损失反馈。
课程价值体现在哪里？	体现 GIS 数据处理、虚拟地理表达、空间过程模拟和交互式决策。

7.3 阶段二：ArcGIS Pro 数据准备与空间基底建立

开发第二阶段重点是把地理数据变成 Unity 可用的数据。该阶段并不是简单下载 DEM 和影像，而是要解决“数据坐标与游戏坐标如何对应”的问题。DEM 提供高程，遥感影像提供纹理和地表参考，ASCII Grid 提供后续洪水扩散的逻辑格网。

在 ArcGIS Pro 中，首先确定 5km x 5km 左右的研究区范围。范围不能太小，否则难以体现洪水扩散和防线布设；也不能太大，否则 Unity 场景对象、玩家移动和格网计算都会变得冗余。最终选择 400 x 400 的格网规模，是为了在精细度和性能之间取得平衡。

- 坐标统一：将 DEM 与遥感影像统一为以米为单位的投影坐标，避免经纬度坐标直接进入 Unity。
- 范围裁剪：使用同一研究区边界裁剪 DEM 与影像，保证二者覆盖范围一致。
- 分辨率处理：将数据处理到约 12.5m 单元格，使 5km 范围对应约 400 x 400 格。
- 配准检查：在 ArcGIS Pro 局部场景中叠加影像与高程，确认河道、坡面和地形起伏方向没有明显错位。
- 格式导出：输出 ASCII Grid 作为 Unity 逻辑数据，同时导出影像纹理用于地表视觉表达。

这一阶段的关键经验是：在 Unity 中看到地形之前，必须先在 GIS 软件中确认数据正确。如果 DEM 与影像在 ArcGIS Pro 中已经不匹配，后续在 Unity 中很难通过代码修正。

7.4 阶段三：Unity 工程搭建与目录组织

进入 Unity 后，首先建立工程目录和主场景。主场景命名为 Assets/Scenes/VGeo_LogicalGrid_5km.unity，表示该场景以 5km 逻辑格网为核心。

脚本集中放置在 Assets/VGeo 目录下，避免与 Unity 自动生成资源、插件资源和临时测试对象混在一起。

工程搭建阶段的重点是先建立“能运行的空框架”：场景能打开、玩家能移动、数据能读取、HUD 能显示基础状态。此时视觉表现可以较简陋，但必须保证后续功能都有挂载对象和脚本入口。

目录/文件	开发作用	说明
Assets/Scenes/VGeo_LogicalGrid_5km.unity	主场景	承载地形、玩家、洪水系统、大坝系统、HUD 和任务对象。
Assets/VGeo/Data/logical_grid_5km.asc	逻辑格网数据	由 GIS 数据处理阶段导出，是洪水扩散和高程判断的基础。
Assets/VGeo/VGeoGridManager.cs	网格管理	负责读取 ASCII Grid、建立格网单元与世界坐标映射。
Assets/VGeo/VGeoFloodSimulator.cs	洪水模拟	负责水位上涨、frontier 扩散、淹没状态更新和大坝阻挡检查。
Assets/VGeo/VGeoBarrierBuilder.cs	大坝建造	负责长坝预览、旋转、预算扣除、实体生成和阻挡边登记。
Assets/VGeo/VGeoHudLayout.cs	HUD 布局	负责预算、洪水、任务、风险和帮助信息显示。

目录组织看似是工程细节，但对课程项目很重要。清晰目录可以让答辩时直接指向关键文件，也让后续检查时能快速说明“数据在哪里、逻辑在哪里、交互在哪里、验证在哪里”。

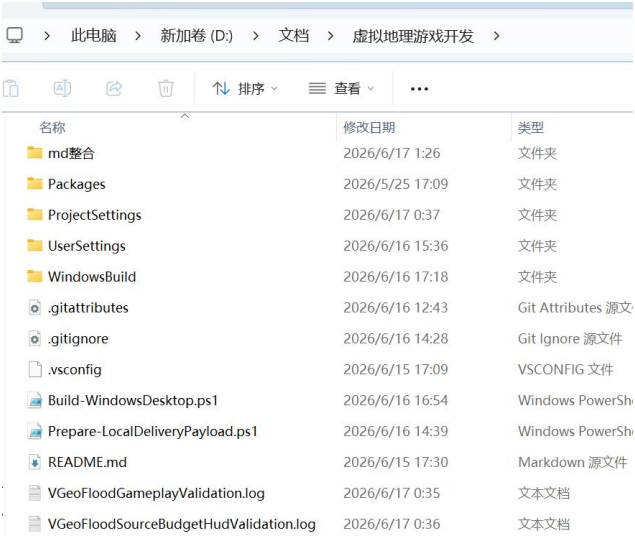


图 6 Unity 项目目录

7.5 阶段四：逻辑格网读取与世界坐标映射

逻辑格网是连接 GIS 数据和 Unity 游戏逻辑的核心。VGeoGridManager 读取 `logical_grid_5km.asc` 后，需要把每个格子的行列号、高程值和世界坐标联系起来。洪水模拟关心的是“某一格是否被淹没”，玩家建坝关心的是“鼠标指向的位置对应哪一格”，HUD 关心的是“当前淹没了多少格”。

开发这一部分时，主要难点在于行列方向、世界坐标方向和 Unity 坐标轴方向可能不一致。如果行列映射反了，洪水源头会出现在错误位置；如果格网缩放不对，12.5m 的格子可能在场景中变成过大或过小；如果高程偏移不对，水面与地形会出现穿插。

- 读取 ASCII Grid 文件头，获得列数、行数、单元大小、无效值和高程矩阵。
- 为每个格子创建逻辑单元，记录 `col`、`row`、`elevation`、`flooded`、`barrier flags` 等状态。
- 通过单元大小计算格网中心点的 Unity 世界坐标。
- 提供 `GridToWorld` 与 `WorldToGrid` 两类接口，支持洪水模拟、鼠标建造和 HUD 查询。
- 在场景中通过 `Flood Source`、玩家出生点和调试输出验证坐标是否正确。

此阶段形成的经验是：洪水模拟能否正常工作，不只取决于算法，还取决于基础格网是否正确。如果一开始没有把数据层做稳，后续大坝、HUD、损失评估都会出现连锁错误。

7.6 阶段五：基础洪水模拟从“瞬时淹没”到“渐进扩散”

洪水模拟最初可以用简单方式验证：当按下启动键后，所有低于目标水位的格子都被标记为淹没。但这种方式虽然容易实现，却不适合演示，因为玩家只会看到结果，而看不到洪水如何从源头扩散。为了增强过程感，后续改为 `frontier` 渐进扩散模型。

`frontier` 模型的思路是：洪水不是一次性出现在所有低洼区，而是从源头格开始，把相邻且满足水位条件的格子逐步加入队列。每次处理一批格子，再等待短暂间隔。这样可以让玩家清楚地看到洪水边界推进，也方便观察大坝阻挡和缺口放水。

迭代版本	实现方式	问题	调整结果
V1: 瞬时淹没	启动后直接遍历全部格网，低于目标水位则标记为淹没。	缺少传播过程，演示效果像静态结果。	改为按源头向外推进。
V2: 逐格扩散	从源头向四邻域扩散。	扩散过慢，且大量格子处理时容易影响帧率。	加入批量处理和时间间隔。
V3: 分批 frontier	每 0.08s 处理最多 18 个格子。	过程更稳定，但需要保证大坝阻挡检查同步。	在相邻格扩散前检查网格边 blocked 状态。
V4: 溃堤续流	沙袋溃堤后重新激活缺口两侧 frontier。	如果不重入队列，缺口放水效果不明显。	洪水可从缺口继续扩散，表现更符合直觉。

最终洪水深度统一为 5m，水位上涨速度为 0.45m/s，扩散批量为 18 cells/frame，扩散间隔为 0.08s。这组参数的目标是让洪水既不会慢到影响汇报节奏，也不会快到玩家看不清过程。

7.7 阶段六：洪水源头固定与启动逻辑修正

洪水源头是整个模拟的起点。开发过程中明确要求：洪水必须从指定 Flood Source 所在格启动，而不是在源头缺失时自动回退到世界原点。这样做的原因是世界原点通常只是 Unity 坐标系统的默认位置，不一定对应河道或低洼区。如果错误回退到原点，洪水路径会失去地理意义。

- 通过列 234、行 142 定位洪水源头，对应世界坐标约为 (437.5, 71.0, 725.0)。
- 洪水启动前检查源头对象和格网数据是否存在。
- 源头缺失时拒绝启动并在 HUD 或日志中提示，而不是默默采用默认位置。
- 玩家出生点设置在 Flood Source + (45, 0, -55) 附近，使进入场景后能较快看到洪水区域。
- 启动洪水前刷新混凝土等关键坝体阻挡边，避免首帧扩散越过防线。

这一阶段解决了演示稳定性问题。固定源头让每次汇报的洪水过程可复现，也让玩家能够围绕同一风险位置反复尝试不同防洪方案。

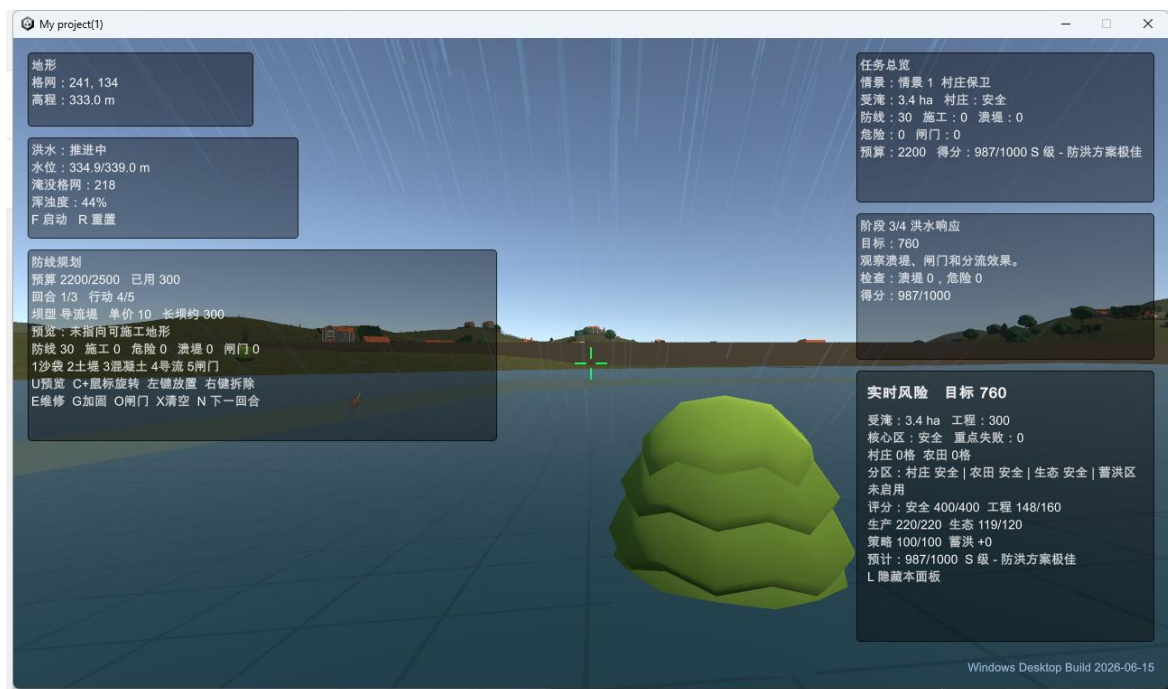


图 7 洪水启动

7.8 阶段七：大坝建造从“摆放模型”到“真正挡水”

大坝系统是开发中最关键的迭代点。单纯在场景中放置一个坝体模型并不能阻止洪水，因为洪水扩散是在逻辑格网中进行的。只有把可见坝体转换为逻辑格网边上的阻挡状态，洪水算法才能在扩散时判断“这两个相邻格之间是否被挡住”。

因此，大坝开发被拆成两层：视觉层负责显示连续坝体，让玩家看到自己建造了什么；逻辑层负责标记被阻挡的网格边，让洪水无法穿过。视觉层和逻辑层必须同步，否则就会出现“看起来有坝但水穿过去”或“看起来没有坝但水被挡住”的问题。

- 鼠标射线命中地形后，将命中点转换为格网行列。
- 根据当前坝型、长度和旋转角度计算长坝覆盖的格网段。
- 显示半透明预览，让玩家在确认前判断位置和方向。
- 左键确认后生成实体坝体，并扣除预算。
- 将坝体覆盖的相邻格边记录为 **blocked**，供洪水扩散时查询。
- 如果坝体被破坏或闸门打开，则更新对应阻挡边状态。

开发问题	表现	解决方式
坝体只是模型	洪水仍会穿过大坝。	将大坝映射为网格边 blocked 状态。
大坝方向不灵活	只能横向或纵向布设，难以贴合地形。	加入按住 C + 鼠标移动连续旋转。
单条坝太短	需要反复建造，汇报操作烦琐。	加长到 30 格，约 375m。
混凝土首帧被穿越	洪水刚启动时未及时读取阻挡边。	启动洪水前刷新混凝土阻挡边。
沙袋缺少材料差异	不同坝型体验不明显。	保留沙袋可破坏和溃堤放水机制。

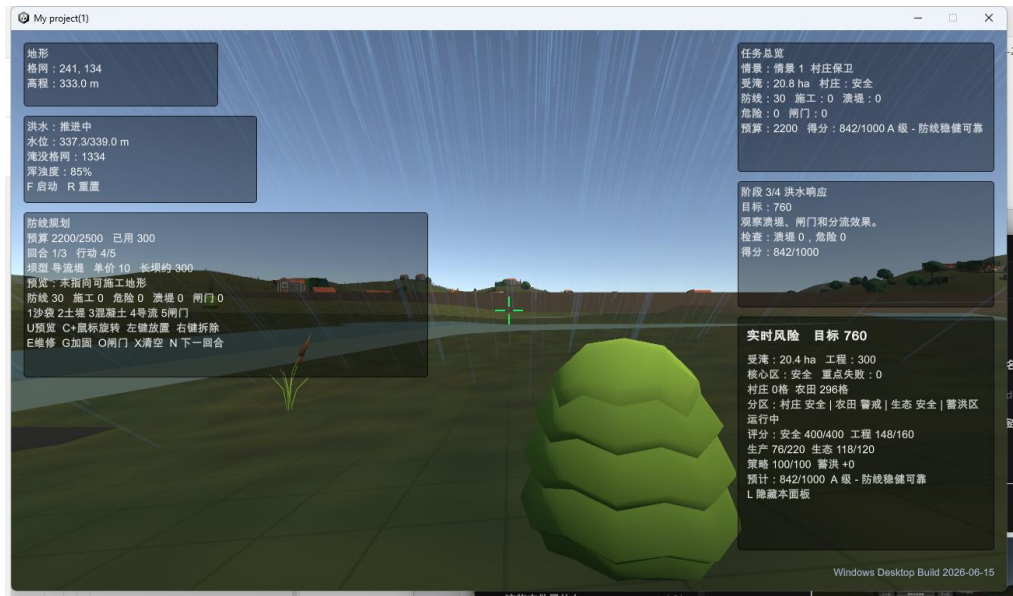


图 8 混凝土坝阻挡洪水

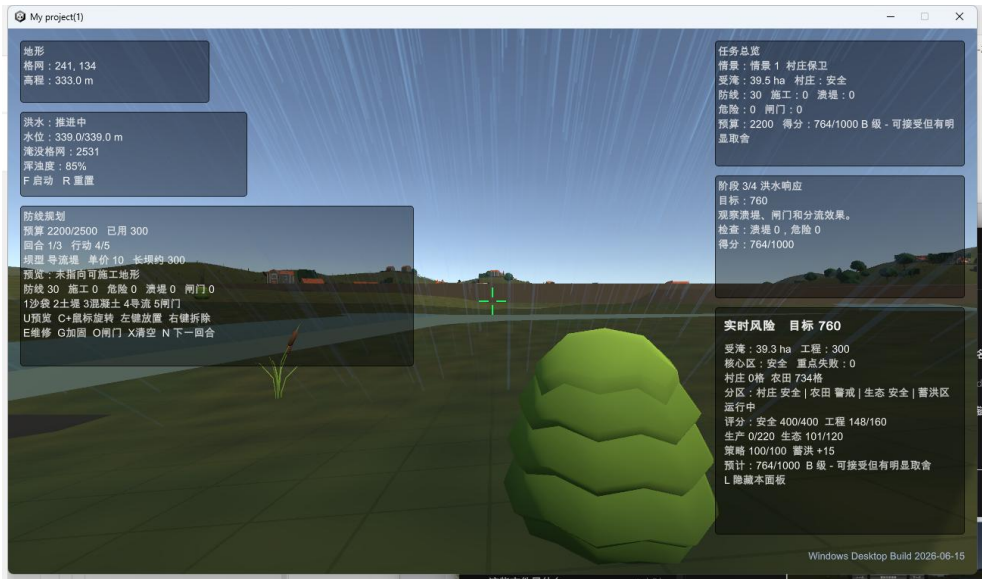


图 9 坝后区域保持未淹没效果

7.9 阶段八：长坝预览、旋转与预算系统

长坝预览是从“功能可用”到“玩家好用”的关键改进。没有预览时，玩家只能点击后才知道大坝位置是否合适，容易误建；加入预览后，玩家可以在确认前观察长度、方向和覆盖范围。

旋转机制最初如果只支持固定角度，会导致大坝难以贴合斜坡、河岸或村庄边界。因此最终采用按住 C 并移动鼠标的连续旋转方式，让玩家可以细调防线方向。该设计更接近实际防洪规划中沿地形布设堤线的思路。

- U 键用于打开或关闭长坝预览，避免默认一直显示造成干扰。
- 预览颜色和实体颜色区分，让玩家知道当前仍处于规划状态。
- 建造前根据坝型和长度估算成本，并在 HUD 中显示。
- 预算不足时不允许建造，防止出现负预算。
- 建造后同时更新实体、逻辑阻挡边、预算和 HUD 状态。

预算系统的调优结果是初始预算 2500。这个数值允许玩家布设有效防线，但不足以无脑铺满整个区域，使防洪策略仍然需要取舍。

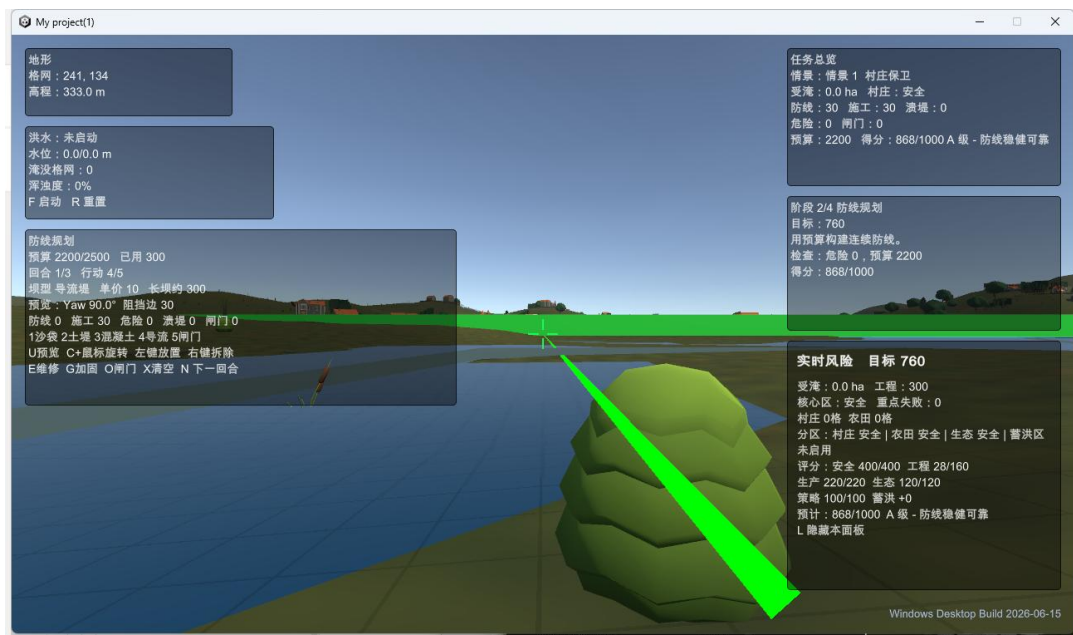


图 10 长坝建成

7.10 阶段九：玩家控制、飞行模式与观察体验

洪水模拟场景的尺度较大，如果只提供普通第一人称移动，玩家很难在短时间内完成巡查和布设。因此开发中对玩家控制进行了针对性增强：移动速度提高到 44，飞行垂直速度设为 26，相机远裁剪距离提高到 24000。

飞行模式的加入主要服务于两类场景：一是玩家需要从高处观察洪水扩散范围和大坝整体效果；二是课程汇报时需要快速展示全局，不适合只在地面慢慢移动。

- 地面移动用于沉浸式巡查，能够从村庄、农田和河岸附近观察风险。
- 飞行模式用于俯瞰和快速定位，适合展示整体洪水路径。
- 玩家出生点靠近源头，减少演示开场寻找洪水位置的时间。
- 远裁剪距离提高后，远处地形、水面和大坝在汇报中更不容易突然消失。
- 水下视觉反馈用于提示玩家进入淹没区域，增强洪水风险感。

7.11 阶段十：HUD 重排与任务引导

HUD 是项目从“程序能运行”走向“观众能看懂”的关键。早期 HUD 如果把预算、洪水状态、任务提示和风险评估都堆在同一位置，会造成信息重叠，汇报时也难以解释。因此后期对 HUD 做了分区重排。

HUD 区域	显示内容	设计理由
左侧信息区	地形格网、洪水状态、防线规划、预算和已用预算。	把玩家建造和洪水状态放在常用视线范围。
右侧任务区	任务总览、阶段提示、风险评估、村庄 / 农田状态。	把解释性信息集中到一侧，便于汇报讲解。
底部中心	水下状态、危险提示。	玩家进入水下或危险区域时能立即看到反馈。
Tab 帮助面板	操作说明和快捷键。	默认隐藏，避免遮挡；需要时再打开。

任务引导让玩家知道当前应该做什么，例如观察风险、布设防线、启动洪水和查看损失。风险反馈则把抽象的淹没格网转化为“村庄是否安全”“农田是否受损”“防线是否危险”等更直观的信息。

7.12 阶段十一：场景真实度与严肃游戏氛围提升

在核心机制稳定后，开发重点转向场景表现。三维地理系统如果只有网格和水面，会更像算法演示；加入村庄、农田、植被、河岸、道路、雨效、音效和水体表现后，玩家更容易把洪水扩散理解为真实风险情境。

- 村庄对象用于表达重点保护目标。
- 农田对象用于表达灾损评估中的经济损失区域。
- 河岸和道路帮助玩家判断洪水可能路径和空间结构。
- 植被与地表纹理增强场景真实感，降低纯技术原型感。
- 雨效和水体效果强化灾害氛围。
- 水下效果用于增强玩家进入洪水区域时的沉浸反馈。

这一阶段的目标不是单纯“美化”，而是让风险对象更容易被识别。课程汇报中，观众不一定了解代码，但能够通过村庄、农田和河岸的空间关系理解为什么要在某些位置布设防线。

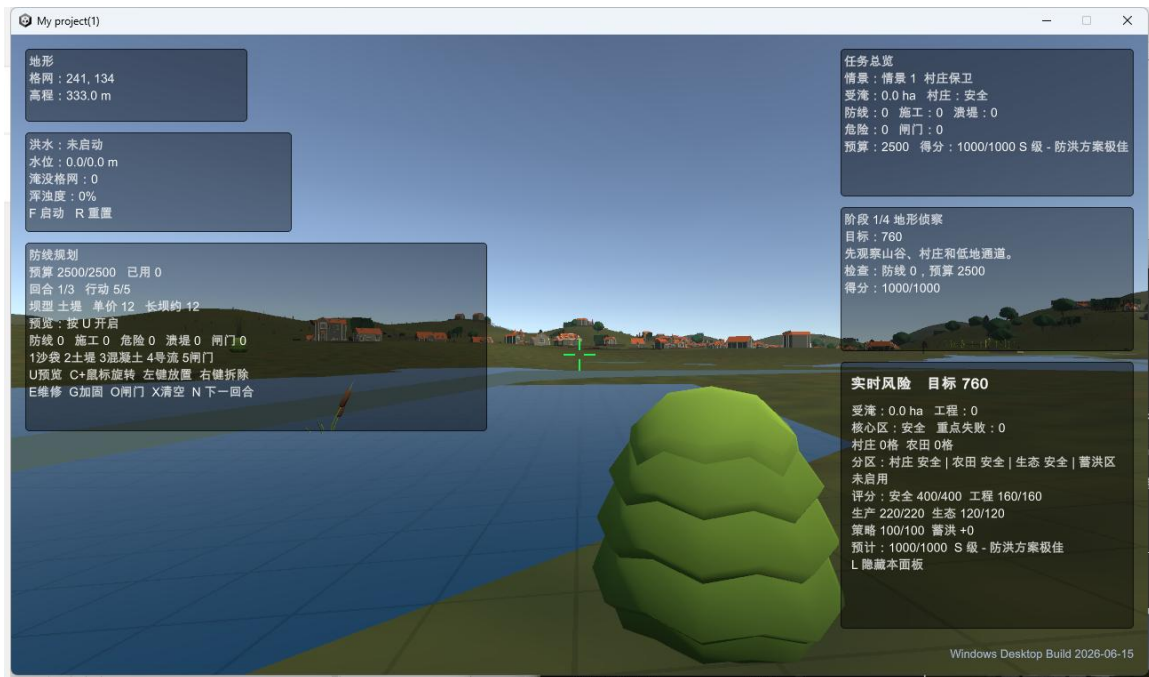


图 11 场景总览

7.13 阶段十二：验证、打包与交付闭环

项目最后阶段重点是从 Unity 编辑器运行转向 Windows 桌面交付。编辑器内能运行并不等于最终交付成功，因为打包时可能出现资源路径、脚本初始化、场景引用和平台设置等问题。因此使用 Build-WindowsDesktop.ps1 和 batch validation 对关键功能进行验证。

- 验证洪水深度是否仍为 5m。
- 验证洪水是否按 18 cells/frame 和 0.08s 间隔推进。
- 验证混凝土大坝是否能在启动首帧前完成阻挡边刷新。
- 验证沙袋溃堤后水能从缺口继续扩散。
- 验证长坝长度是否为 30 格。
- 验证 HUD 关键区域是否不重叠。
- 验证 WindowsBuild 输出是否成功，并检查日志是否包含 Build Finished, Result: Success。

这一阶段形成了完整闭环：GIS 数据完成输入，Unity 完成运行逻辑，玩家完成交互，HUD 完成反馈，验证脚本完成检查，WindowsBuild 完成交付。

第八章 开发难点、问题修复与参数迭代

8.1 主要开发难点

本项目的难点不在单个脚本，而在多个系统之间的联动。地形数据、逻辑格网、洪水扩散、大坝建造、玩家控制和 HUD 展示都必须使用同一套空间逻辑。如果某个环节参数不一致，最终表现就会出现明显问题。

难点	原因	解决思路
GIS 数据与 Unity 坐标对齐	GIS 坐标、格网行列和 Unity 世界坐标存在方向与尺度转换。	统一投影坐标、固定格网单元大小，并通过源头和出生点反复校验。
洪水既要可见又要稳定	过快看不清过程，过慢影响汇报节奏；一次性遍历又缺少过程感。	采用分批 frontier 扩散和固定时间间隔。
大坝视觉与逻辑同步	模型本身不会阻止格网扩散。	将大坝覆盖位置转化为 blocked 网格边。

难点	原因	解决思路
多坝型差异表达	如果所有坝都一样，决策机制会变弱。	设置沙袋、土堤、混凝土、导流堤和闸门的不同定位。
HUD 信息量较大	预算、洪水、任务、风险、帮助信息容易重叠。	左侧、右侧、底部分区，帮助面板默认隐藏。
最终交付稳定性	编辑器运行与桌面程序运行存在差异。	使用验证脚本和 WindowsBuild 形成交付检查。

8.2 关键问题修复记录

问题	现象	定位过程	修复结果
洪水源头不稳定	洪水可能从不符合预期的位置启动。	检查启动逻辑，发现需要明确处理源头缺失情况。	固定源头格网列 234、行 142；源头缺失时拒绝启动。
大坝首帧失效	混凝土坝明明存在，但洪水启动瞬间可能越过。	判断为阻挡边刷新时机晚于洪水扩散首帧。	洪水启动前刷新混凝土阻挡边。
沙袋溃堤不明显	沙袋破坏后水没有明显从缺口继续流出。	发现缺口附近未重新进入 frontier 队列。	溃堤后把缺口两侧重新加入传播队列。
长坝覆盖不足	玩家需要连续点击多次才能形成有效防线。	检查单条坝长度与 5km 场景尺度不匹配。	单条长坝加长到 30 格，约 375m。
HUD 信息重叠	预算、任务和风险文字互相遮挡。	检查 UI 锚点和面板分区。	左侧、右侧、底部重排，Tab 帮助默认隐藏。
演示视野受限	远处地形和水面不易观察。	检查相机裁剪距离和玩家移动速度。	远裁剪提升到 24000，移动和飞行速度上调。

8.3 参数迭代说明

项目参数不是孤立设置，而是服务于演示节奏和玩法平衡。洪水深度、水位上涨速度、扩散批量、长坝长度和预算共同决定玩家看到的结果。

参数	最终值	调优理由
洪水深度	5m	能够产生明显淹没效果，同时让混凝土防线有可靠阻挡空间。
水位上涨速度	0.45m/s	让水位变化可观察，不至于瞬间完成。
扩散批量	18 cells/frame	兼顾传播速度与帧率稳定。
扩散间隔	0.08s	让洪水推进具有节奏感，便于演示讲解。

参数	最终值	调优理由
初始预算	2500	既能建设有效防线，又保留策略取舍。
长坝长度	30 格	减少重复操作，使单条防线具备空间意义。
玩家速度	44	适应 5km 场景尺度，提高巡查效率。
相机远裁剪	24000	避免汇报时远处场景过早裁剪。

8.4 可解释性设计

由于项目用于课程汇报，很多技术设计都需要能够被非开发者理解。离散格网扩散虽然不等同于真实水动力模型，但它有很强的解释性：每个格子有高程，水位决定能否淹没，frontier 决定传播边界，大坝决定相邻格之间是否能通过。

这种可解释性让项目更容易在答辩中说明：如果水绕过大坝，是因为防线没有完全闭合；如果沙袋破坏后水继续传播，是因为缺口两侧重新加入了扩散队列；如果混凝土堤能稳定阻挡，是因为其阻挡边和有效坝顶在 5m 洪水玩法下被设定为可靠。

第九章 测试与验证

项目包含 Unity batch validation，用于在不手动打开编辑器交互的情况下验证关键功能。每次关键改动后，均运行验证并重新构建桌面程序。

验证项	通过标准	开发意义
洪水深度	最终水深为 5m，并在模拟与汇报材料中保持一致。	防止文档、PPT 与程序参数不一致。
渐进扩散	洪水按 18 cells/frame、0.08s 间隔分批推进。	保证洪水过程可观察且运行稳定。
混凝土阻挡	启动首帧前刷新阻挡边，可阻挡洪水且不会被常规洪水自动摧毁。	验证主要防线机制可靠。
沙袋机制	沙袋能够阻挡洪水，并在溃堤后允许水流从缺口继续扩散。	验证材料差异和溃堤反馈。
长坝长度	单条长坝为 30 格，约 375m，并在模拟与汇报材料中保持一致。	保证玩家建造体验和文档说明一致。
HUD 布局	预算、源头、任务、风险和水下状态不重叠。	保证演示时关键信息可读。
桌面构建	WindowsBuild 输出成功，日志包含 Build Finished, Result:	证明项目可独立交付运行。

验证项	通过标准	开发意义
	Success.	

9.1 功能测试流程

1. 打开主场景，确认 VGeoGridManager 正确读取 logical_grid_5km.asc。
2. 检查玩家出生点是否位于洪水源头附近并贴合地形。
3. 按 U 打开长坝预览，按住 C 旋转，确认预览方向连续变化。
4. 选择混凝土大坝并布设在洪水传播路径上，确认预算扣除和 HUD 更新。
5. 按 F 启动洪水，观察水位上涨和 frontier 分批扩散。
6. 检查洪水遇到混凝土大坝是否被阻挡，是否从未封闭缺口绕行。
7. 布设沙袋并触发溃堤，确认缺口附近洪水继续传播。
8. 切换飞行模式，从高处观察洪水范围、村庄安全和农田影响。
9. 查看任务提示、损失评估和最终反馈是否与实际淹没状态一致。

9.2 打包验证流程

1. 保存主场景和关键资源，确保构建场景列表包含 VGeo_LogicalGrid_5km.unity。
2. 运行 Build-WindowsDesktop.ps1，启动 Unity batch 构建。
3. 检查构建日志，确认没有脚本编译错误、资源缺失或场景引用错误。
4. 确认输出目录中生成 VGeoFloodRiskDemo.exe。
5. 脱离 Unity 编辑器直接运行 exe，检查玩家控制、HUD、建坝和洪水启动是否正常。
6. 如发现桌面程序表现与编辑器不同，优先检查资源路径、初始化顺序和场景对象引用。

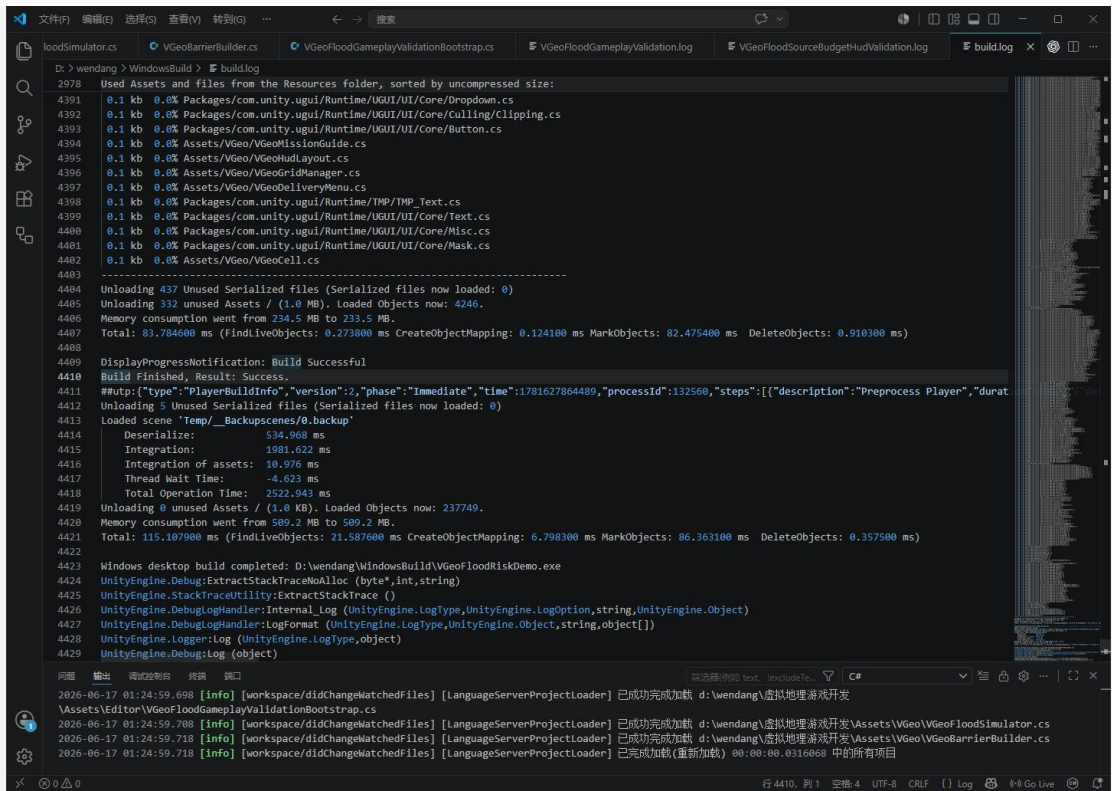


图 12 构建成果日志

9.3 验证结论

最终版本已完成 30 格连续长坝、混凝土可靠阻挡、沙袋溃堤放水、固定洪水源头、2500 初始预算、HUD 重排和 Windows 桌面程序交付。

第十章 项目成果与价值

10.1 项目成果

- 完成可运行 Unity 主场景与 Windows 桌面程序。
- 完成从 GIS 数据准备到 Unity 逻辑格网的空间数据接入流程。
- 完成洪水源头、渐进扩散、水位上涨和地形约束机制。
- 完成连续长坝、网格边阻挡、混凝土可靠防线、沙袋溃堤放水和闸门控制。
- 完成 HUD、任务引导、风险评估、损失统计和水下视觉反馈。
- 形成较完整的开发过程记录，可说明项目从原型到交付的迭代路线。

10.2 应用价值

《洪线之上》的价值在于把地理数据、三维场景、灾害模拟和游戏交互融合到同一个可运行系统中。它既可以作为虚拟地理课程汇报作品，也可以作为洪水风险认知和防洪决策训练的严肃游戏原型。

- 虚拟地理教学价值：帮助学习者理解地形格网、空间配准和三维场景间关系。
- 洪水风险认知价值：让洪水蔓延、地形高差和风险对象之间的关系变得可观察。
- 防洪决策训练价值：通过预算、坝型和损失反馈，引导玩家进行策略比较。
- 工程实践价值：形成从数据处理、系统开发、验证到桌面交付的完整闭环。
- 课程答辩价值：开发过程、问题修复和参数迭代均可作为汇报亮点。

10.3 与普通三维场景的区别

普通三维场景主要强调可视化展示，而《洪线之上》加入了可交互的空间过程。玩家看到的不只是地形和建筑，还能通过建坝改变洪水传播路径，并通过 HUD 得到损失评估。这使项目从“展示型场景”升级为“交互型虚拟地理系统”。

第十一章 总结

当前版本作为最终交付系统，具备完整的演示流程、稳定的核心机制、可解释的技术路径和可运行的桌面程序，展现出“GIS 数据准备 + 虚拟地理 + 洪水模拟 + 防洪决策 + 游戏化交互 + 完整交付”的全过程及组合价值。至此，《洪线之上》完成了从选题构想到 GIS 数据处理、Unity 场景构建、洪水模拟、防洪工程交互、HUD 与损失评估、自动验证和 Windows 桌面交付的完整开发闭环。

第十二章 小组分工与协作

12.1 小组成员与总体协作方式

本项目由六名成员共同完成，整体采用“主责分工 + 交叉协作”的方式推进。由于《洪线之上》同时包含 GIS 数据处理、Unity 场景搭建、C# 代码开发、交互体验设计、测试验证、文档整理和汇报展示等任务，单一成员很难独立覆盖全部工作，因此小组将任务拆分为若干相互衔接的模块。

在协作过程中，前期由整体规划成员确定作品主题、系统目标和功能边界；数据准备成员负责将 DEM、遥感影像和 ASCII Grid 等资料处理为 Unity 可用的数据基础；场景与代码成员分别推进三维环境和核心逻辑；交互与测试成员负责把功能转化为可演示流程；文档与汇报成员则将开发过程、技术细节和最终成果整理为课程提交材料。各成员虽然有主责方向，但在关键节点会共同参与功能讨论、问题排查和最终演示检查。

12.2 具体成员分工

成员	主要分工	具体工作内容	主要产出
	整体规划与组内统筹	负责项目选题、目标定位、功能框架和开发路线设计；梳理“GIS 数据准备 - Unity 场景 - 洪水模拟 - 防洪决策 - 损失评估 - 桌面交付”的整体链路；协调各成员进度，整理阶段任务清单，并在最终汇报中把握作品叙事逻辑。	项目总体方案、功能模块划分、阶段任务安排、汇报主线与演示流程。
	前期准备与 GIS 数据处理	负责研究区范围确定、DEM 与遥感影像资料整理、坐标系统一、裁剪、重采样、配准检查和 ASCII Grid 导出；协助记录 ArcGIS Pro 处理步骤，并将 400 x 400、约 12.5m 单元格的逻辑格网参数统一到说明书与 PPT 中。	数据处理流程、DEM / 影像基础资料、logical_grid_5km.asc 数据说明、GIS 处理记录。
	Unity 场景搭建与空间表达	负责 Unity 主场景的地形导入、影像贴图、村庄、农田、河岸、道路、植被、水体和环境氛围布置；根据洪水源头和玩家观察路线调整场景表现，使作品从单纯功能原型转化为可巡查、可演示的虚拟地理场景。	VGeo_LogicalGrid_5km.unity 场景基础、地物布置、环境美化、展示截图素材。
	核心代码开发与洪水逻辑	负责 C# 核心脚本开发，重点包括 VGeoGridManager 格网读取与行列映射、VGeoFloodSimulator 洪水源头、水位上涨和 frontier 扩散逻辑、网格边阻挡判断、混凝土坝可靠阻挡、沙袋溃堤后续流等关键机制。	格网管理、洪水模拟、大坝阻挡、溃堤续流等核心代码模块。
	交互开发、HUD 与测试验证	负责玩家控制、飞行观察、长坝预览、连续旋转、坝型切换、预算显示、任务提示、风险评估和损失统计等交互反馈内容；参与 HUD 重排、参数调优和 batch validation 验证，重点保证系统“能玩、能看、能解释”。	FirstPersonGeoCamera、HUD 布局、任务引导、损失评估、测试记录与问题修复记录。
	文档处理、汇报制作与交付整理	负责项目报告书、PPT、操作说明、成果截图和最终材料整理；对技术参数、文件路径、构建结果和演示步骤进行统一校对；协助打包检查，保证提交材料中工程文件、可执行程序 and 文字说明能够对应起来。	项目报告书、汇报 PPT、演示讲稿、WindowsBuild 交付说明、最终文件索引。

12.3 阶段性协作分工

开发阶段	主责与协作	阶段任务说明
选题与需求阶段		确定项目名称、应用定位、严肃游戏方向和最终演示目标，明确洪水风险、防洪决策和灾损评估三条主线。
数据准备阶段		完成研究区数据整理、DEM 和影像处理、ASCII Grid 导出，并确认地形高程、纹理范围和 Unity 场景尺度能够对应。
场景搭建阶段		将地理数据转化为可观察场景，补充村庄、农田、道路、河岸、植被、水体和环境效果，为后续洪水模拟提供空间语境。
核心功能阶段		实现洪水源头、5m 水深、水位上涨、frontier 扩散、大坝阻挡、长坝建造、沙袋溃堤和混凝土防线等核心逻辑。
交互优化阶段		优化第一人称与飞行观察体验，调整 HUD 信息层级、任务提示、预算显示和损失反馈，解决遮挡、参数不直观和演示路径不清晰等问题。
测试交付阶段		整理构建脚本和文件路径，检查 Windows 桌面程序能否运行，围绕洪水扩散、混凝土阻挡、沙袋溃堤、长坝长度和 HUD 布局进行最终验证。

12.4 小组协作中的问题处理

小组协作中最容易出现的问题是不同模块之间的参数不统一。例如，GIS 数据阶段关注的是 DEM 分辨率、投影坐标和 ASCII Grid 行列数，Unity 开发阶段关注的是世界坐标、格网尺寸和场景比例，文档阶段则需要把这些技术参数转化为可读的说明。为避免出现“程序里一个参数、说明书里另一个参数”的情况，小组在后期统一将最终版本描述为 400 x 400、约 12.5m 单元格、5m 洪水深度、30 格长坝和 2500 初始预算。

另一个典型问题是功能可以运行但不一定适合演示。早期洪水扩散如果过快，观众很难看清水位上涨和防线阻挡；玩家出生点如果离源头太远，汇报时会浪费时间寻找关键位置；HUD 如果堆在同一侧，预算、任务和风险信息容易重叠。针对这些问题，小组通过调整扩散间隔、玩家出生位置、相机远裁剪距离、HUD 分区和帮助面板显示方式，使最终版本更适合课堂汇报与现场演示。

最终形成的协作方式不是简单地把任务平均分给六个人，而是围绕项目链路进行衔接：数据处理为场景和逻辑格网提供基础，场景搭建为洪水演进提供可观察环境，代码开发提供核心玩法，交互反馈让玩家理解结果，测试验证保证系统稳定，文档汇报则把开发过程和成果价值完整表达出来。

第十三章 小组成员心得体会

13.1 心得体会

作为整体规划与统筹负责人，我最大的体会是课程项目不能只停留在想法上，而要把想法拆解成可以落地的模块。《洪线之上》最初只是“洪水模拟游戏”的方向，但真正推进时需要明确研究区数据、地形网格、洪水源头、大坝机制、玩家操作、HUD 反馈和最终交付形式。先把系统边界想清楚，后续成员分工和功能实现才不会出现偏差。

在项目过程中，我也认识到虚拟地理作品的重点不只是画面好看，更重要的是把地理过程讲清楚。洪水为什么从这个源头扩散、为什么会被大坝挡住、为什么预算会影响防线选择，这些内容都需要通过系统设计和汇报逻辑表达出来。本次项目让我对“地理数据 + 游戏交互 + 决策模拟”的结合方式有了更具体的理解。

13.2

我主要负责前期 GIS 数据准备。通过这次项目，我更加直观地感受到数据处理对后续开发的重要性。DEM、遥感影像和 ASCII Grid 看起来只是前期材料，但如果坐标系统、裁剪范围或分辨率处理不一致，导入 Unity 后就会出现地形比例不对、影像对不上、格网行列难以解释等问题。

以前学习 ArcGIS Pro 时，更多是完成单个工具操作；这次项目则要求把数据处理结果真正交给程序使用。为了让说明书和程序一致，我需要关注格网、单元格、世界坐标映射等细节。这个过程让我明白 GIS 数据不是孤立存在的，它可以成为三维场景、模拟算法和交互系统的基础。

13.3

我主要参与 Unity 场景搭建与空间表达。这个部分看似偏向美术，但实际与地理表达关系很密切。村庄、农田、道路、河岸和植被如果只是随意摆放，玩家就很难理解洪水风险对象之间的空间关系；只有让这些对象与地形起伏、河道位置和洪水扩散方向联系起来，场景才具有虚拟地理意义，

通过该项目我体会到：场景搭建不是把资源堆上去，而是服务于演示目标。比如玩家一进入场景应该能较快判断哪里是低洼区、哪里可能被洪水影响、哪里适合布设防线。后期在环境氛围、视角路线和截图素材整理中，我也认识到一个课程作品要想让别人看懂，视觉呈现和空间逻辑必须相互配合。

13.

我主要负责核心代码开发，尤其是逻辑格网、洪水扩散和大坝阻挡相关内容。开发过程中最深的体会是，模拟效果不一定要追求复杂，但规则必须清晰、稳定并且可解释。本项目没有采用完整二维水动力模型，而是使用离散格网和 **frontier** 扩散方式。这样既能保证运行效率，又能让玩家看清洪水逐步蔓延的过程。

在调试大坝系统时，我认识到视觉表现和逻辑判断是两件事。屏幕上看到一条连续长坝，并不代表程序中自然就能挡水；必须把长坝转换成格网边阻挡，并在洪水扩散时逐格检查相邻边是否被 **blocked**。混凝土坝首坝阻挡、沙袋溃堤后缺口续流等问题，让我对 **Unity** 中“可见对象”和“运行逻辑”之间的关系有了更深理解。

13.5

我主要负责交互、HUD 和测试验证。通过这次项目，我体会到一个系统“能运行”和“好使用”之间还有很大距离。早期版本虽然已经有洪水扩散和大坝建造，但玩家不一定知道当前预算是多少、选择了哪种坝型、洪水是否已经启动、村庄和农田是否安全。因此 HUD、任务提示和风险反馈对理解系统非常关键。

测试过程中，我也认识到参数调优需要围绕演示体验进行。扩散太快会看不清过程，扩散太慢会影响课堂展示；长坝太短会让玩家反复建造，太长又可能削弱策略选择；HUD 信息太多会遮挡画面，太少又无法解释结果。最终通过多轮调整，我对交互设计、可视化反馈和功能验证之间的关系有了更具体的认识。

13.6

我主要负责文档处理、汇报制作和最终材料整理。这个过程让我认识到，项目成果不仅要做出，还要能够被清楚地说明。**Unity** 工程、脚本文件、构建程序、数据路径和测试结果如果没有整理成系统化文档，别人很难理解项目完成了哪些功能、采用了哪些技术路线、解决了哪些问题。

在整理说明书和 **PPT** 的过程中，我特别关注参数、截图和表述是否一致。例如洪水深度、长坝长度、预算数值、主场景路径和构建结果都需要统一，否则答辩时容易出现前后矛盾。通过这次项目，我体会到文档不是最后简单补上的材料，而是对开发过程的复盘和表达，也是把小组技术工作转化为课程成果的重要环节。

附录：关键文件索引

类别	文件路径	说明
主场景	Assets/Scenes/VGeo_LogicalGrid_5km.unity	最终演示场景。
网格管理	Assets/VGeo/VGeoGridManager.cs	读取 ASCII Grid，管理网格与世界坐标映射。
洪水模拟	Assets/VGeo/VGeoFloodSimulator.cs	实现水位上涨、frontier 扩散和淹没状态更新。
大坝建造	Assets/VGeo/VGeoBarrierBuilder.cs	实现长坝预览、旋转、预算和阻挡边登记。
玩家相机	Assets/VGeo/FirstPersonGeoCamera.cs	实现第一人称移动与飞行观察。
HUD 布局	Assets/VGeo/VGeoHudLayout.cs	组织预算、洪水、任务和风险信息显示。
任务引导	Assets/VGeo/VGeoMissionGuide.cs	管理阶段提示和任务引导。
损失评估	Assets/VGeo/VGeoLossAssessment.cs	统计淹没范围、村庄、农田和防汛结果。
场景导演	Assets/VGeo/VGeoScenarioDirector.cs	组织场景启动与演示流程。
水下效果	Assets/VGeo/VGeoUnderwaterEffect.cs	实现玩家进入水下时的视觉反馈。
验证脚本	Assets/Editor/VGeoFloodGameplayValidationBootstrap.cs	用于批处理验证关键参数和机制。
构建脚本	Build-WindowsDesktop.ps1	用于 Windows 桌面程序构建。